

INTEGER TYPES

Picking the right size for a whole number

THE QUESTION

How big can this number realistically get?

Not every whole number in the tracker needs the same amount of room - picking a type that's too small overflows, too large just wastes memory.

SHORT, INT, LONG LONG

Three sizes, three jobs

```
short expensesLoggedToday{7};
int transactionId{1'000'000};
long long lifetimeCentsProcessed{9'000'000'000};
```

VARIABLE	TYPE	WHY
expensesLoggedToday	short	small, everyday count
transactionId	int	the natural default
lifetimeCentsProcessed	long long	climbs into the billions

CHECKING SIZES

sizeof tells you the truth

```
std::println("expensesLoggedToday    : {} ({} bytes)",
             expensesLoggedToday, sizeof(expensesLoggedToday));
std::println("transactionId          : {} ({} bytes)",
             transactionId, sizeof(transactionId));
std::println("lifetimeCentsProcessed : {} ({} bytes)",
             lifetimeCentsProcessed, sizeof(lifetimeCentsProcessed));
```

sizeof reports how many bytes a type actually occupies on this platform/compiler.

UNSIGNED

When negative can't happen

```
unsigned int receiptsOnFile{250};
```

Some numbers - like a count of receipts on file - can never sensibly go negative. Marking the type **unsigned** trades away negative values for a bit more positive range out of the same bytes.

```
// unsigned int negativeAttempt{-5}; // compiler error
```

THE CLASSIC BUG

Unsigned subtraction wraps around

```
unsigned int receiptsAfterVoiding{receiptsOnFile - 300};  
// receiptsOnFile is 250, so this "should" be -50...
```

Unsigned integers never go negative - they wrap around instead, silently. This is a classic source of expense-tracker bugs: a total that should be negative becomes an enormous positive number.

<stdint> fixed-width types

```
#include <stdint>

std::int16_t compactExpenseCount{7};
std::int32_t portableTransactionId{1'000'000};
std::uint32_t portableReceiptsOnFile{250};
```

short / **int** / **long** **long** describe a size, but the exact number of bits those keywords map to can differ across platforms and compilers.

<stdint> gives an exact, portable width instead - the modern way to say "I need exactly this many bits" (e.g. for a save-file format).

DEBUG IT

Watch the wraparound happen

Breakpoint on the `receiptsAfterVoiding` line and step over it - watch the value jump to a huge positive number instead of going negative. Seeing it happen once in the debugger makes the warning stick.